Actividad 6. K vecinos más cercanos

Primero se cargó la base de datos en Python y se revisó su contenido. Se encontraron 14,998 observaciones y 10 columnas. Entre las variables se encuentran el nivel de satisfacción, el resultado de la última evaluación, el número de proyectos, las horas promedio trabajadas al mes, el tiempo en la empresa, los accidentes laborales, los ascensos recientes, la variable que indica si el empleado se fue, el departamento y categoría de salario.

```
[86]: import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt
      import os
      os.chdir("D:/gus6e/Documents")

[87]: data = pd.read_csv("recursos_humanos.csv")
      data
```

[87]:

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident	left	promotion_last_5years	sales	salary
0	0.38	0.53	2	157	3	0	1	0	sales	low
1	0.80	0.86	5	262	6	0	1	0	sales	medium
2	0.11	0.88	7	272	4	0	1	0	sales	medium
3	0.72	0.87	5	223	5	0	1	0	sales	low
4	0.37	0.52	2	159	3	0	1	0	sales	low
...	...	...	...	...	...	...	...	...	...	...
14994	0.40	0.57	2	151	3	0	1	0	support	low
14995	0.37	0.48	2	160	3	0	1	0	support	low
14996	0.37	0.53	2	143	3	0	1	0	support	low
14997	0.11	0.96	6	280	4	0	1	0	support	low
14998	0.37	0.52	2	158	3	0	1	0	support	low

14999 rows × 10 columns

A diferencia de la actividad anterior, aquí sí quitamos las variables que no se mencionan en las diapositivas (para ser dogmáticos con lo pedido), y designamos las categorías a la variable “left”, para los casos positivos de dejar el trabajo (1) y los que permanecieron (0).

```
[88]: data.drop(["number_project"], axis = 1, inplace = True)
      data.head()
```

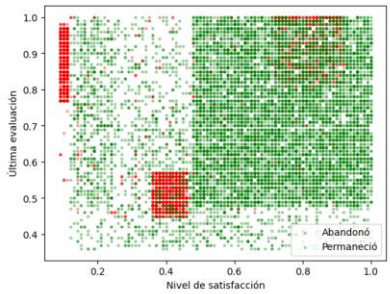
[88]:

	satisfaction_level	last_evaluation	average_montly_hours	time_spend_company	Work_accident	left	promotion_last_5years	sales	salary
0	0.38	0.53	157	3	0	1	0	sales	low
1	0.80	0.86	262	6	0	1	0	sales	medium
2	0.11	0.88	272	4	0	1	0	sales	medium
3	0.72	0.87	223	5	0	1	0	sales	low
4	0.37	0.52	159	3	0	1	0	sales	low

```
[89]: A = data[data.left == 1] # Abandonó
      P = data[data.left == 0] # Permaneció
```

Antes de entrenar KNN se separaron los empleados en dos categorías para observar algunas de sus características. Se elaboró un diagrama de dispersión utilizando el nivel de satisfacción y el resultado de la última evaluación, aquí también se bajó el tamaño de las observaciones porque era poco entendible, usando “s = 5”, para disminuir el tamaño. Finalmente, usando una función para contar los valores de left, para ambos casos, vemos que no está balanceada, ya que es muy diferente la proporción de personas que dejaron con las que permanecieron, siendo de 11428 y 3571 respectivamente

```
plt.xlabel("Nivel de satisfaccion")
plt.ylabel("Última evaluación")
plt.legend()
plt.show()
```




```
# Número de observaciones de cada categoría
data.left.value_counts()
```

```
left
0    11428
1     3571
```

Se utilizaron las instrucciones pd.get_dummies de las instrucciones para transformar las variables cualitativas que aparecen en las columnas sales y salary, como lo sugiere la actividad. En lugar de dejar el departamento y el salario escritos como palabras, se generaron columnas numéricas que indican a qué categoría pertenece cada trabajador. Después de la transformación se obtuvieron 20 características predictoras. Siguiendo el procedimiento de la Lección 6, se separó la variable que se desea predecir del conjunto de características.

```
# Recodificación de las variables cualitativas con pd.get_dummies
data = pd.get_dummies(data, columns = ["sales", "salary"], dtype = int)
data.head()
```

	satisfaction_level	last_evaluation	average_montly_hours	time_spend_company	Work_accident	left	promotion_last_5years	sales_IT	sales_RandD	sales_accounting
0	0.38	0.53	157	3	0	1	0	0	0	0
1	0.80	0.86	262	6	0	1	0	0	0	0
2	0.11	0.88	272	4	0	1	0	0	0	0
3	0.72	0.87	223	5	0	1	0	0	0	0
4	0.37	0.52	159	3	0	1	0	0	0	0

```
y = data.left.values
x_data = data.drop(["left"], axis = 1)
```

```
y
```

```
array([1, 1, 1, ..., 1, 1, 1], shape=(14999,))
```

Después se aplicó la normalización que aparece en el código de clase para trabajar con los valores numéricos antes de entrenar el modelo.

```
# Normalización
x = (x_data - np.min(x_data)) / (np.max(x_data) - np.min(x_data))
x
```

	satisfaction_level	last_evaluation	average_montly_hours	time_spend_company	Work_accident	promotion_last_5years	sales_IT	sales_RandD	sales_accounting	s
0	0.001226	0.001710	0.506452	0.009677	0.0	0.0	0.0	0.0	0.0	
1	0.002581	0.002774	0.845161	0.019355	0.0	0.0	0.0	0.0	0.0	
2	0.000355	0.002839	0.877419	0.012903	0.0	0.0	0.0	0.0	0.0	
3	0.002323	0.002806	0.719355	0.016129	0.0	0.0	0.0	0.0	0.0	
4	0.001194	0.001677	0.512903	0.009677	0.0	0.0	0.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	
14994	0.001290	0.001839	0.487097	0.009677	0.0	0.0	0.0	0.0	0.0	
14995	0.001194	0.001548	0.516129	0.009677	0.0	0.0	0.0	0.0	0.0	

Después, se dividieron los registros en dos partes: 70 % para entrenar KNN y 30 % para evaluar sus predicciones. Se conservó la misma configuración de selección aleatoria utilizada en la lección, de modo que la comparación entre los diferentes valores de K se realizara con los mismos datos de prueba.

14999 rows × 19 columns

```
# Bases de entrenamiento y prueba
from sklearn.model_selection import  train_test_split
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size = 0.30, random_state = 1)
```

Primero se ajustó el modelo con tres vecinos, siguiendo el ejemplo original. Posteriormente se repitió el entrenamiento para cada valor de K desde 1 hasta 20, como indica la actividad. Para comparar las alternativas se elaboró una tabla con la precisión global, la precisión de las predicciones de permanencia y la precisión de las predicciones de abandono.

```
# Modelo KNN (K vecinos más cercanos)
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors = 3)
knn.fit(x_train,y_train)
prediction = knn.predict(x_test)
print("Score: ", knn.score(x_test,y_test))
```

```
Score:  0.8888888888888888
```

```
# Creación de la tabla de comparación para k = 1, 2, ..., 20
from sklearn.metrics import confusion_matrix
resultados = []
for k in range(1,21):
    knn = KNeighborsClassifier(n_neighbors = k)
    knn.fit(x_train,y_train)
    prediction = knn.predict(x_test)
    y_pred = prediction
    y_true = y_test
    cm = confusion_matrix(y_true,y_pred)
    correctos = cm[0,0] + cm[1,1]
    incorrectos = cm[0,1] + cm[1,0]
    precisionglobal = correctos / (correctos + incorrectos)
    precisionpermanece = cm[0,0] / (cm[0,0] + cm[1,0])
    precisionabandona = cm[1,1] / (cm[1,1] + cm[0,1])
    resultados.append([k,precisionglobal,precisionpermanece,precisionabandona])

tabla = pd.DataFrame(resultados, columns = ["K", "Precisión global", "Precisión permanece", "Precisión abandona"])
tabla
```

El valor K=1 alcanzó la precisión global más alta de los veinte modelos: aproximadamente 91.04 %. Además, su precisión al pronosticar que un empleado permaneció fue de 96.05 %, mientras que la precisión al pronosticar que abandonó la empresa fue de 77.70 %. K=2 fue el segundo con mejor precisión global casi al primer vecino y la precisión de abandono más alta, pero se prefirió usar el primero por tener un poco más de precisión global.

	K	Precisión global	Precisión permanece	Precisión abandona
0	1	0.910444	0.960563	0.777055
1	2	0.904889	0.929804	0.820312
2	3	0.888889	0.953640	0.727061
3	4	0.892667	0.931959	0.771946
4	5	0.886222	0.950372	0.724138
5	6	0.895333	0.938114	0.769096
6	7	0.890222	0.949815	0.735623
7	8	0.897333	0.939583	0.772807
8	9	0.896667	0.954140	0.747402
9	10	0.900222	0.946434	0.769754
10	11	0.894667	0.954855	0.740916

Se volvió a ajustar el modelo con K = 1 y se elaboró el mapa de calor de la matriz de confusión, tal como se hizo en la lección. En las filas se muestran los resultados reales de los empleados y en las columnas las predicciones del modelo.

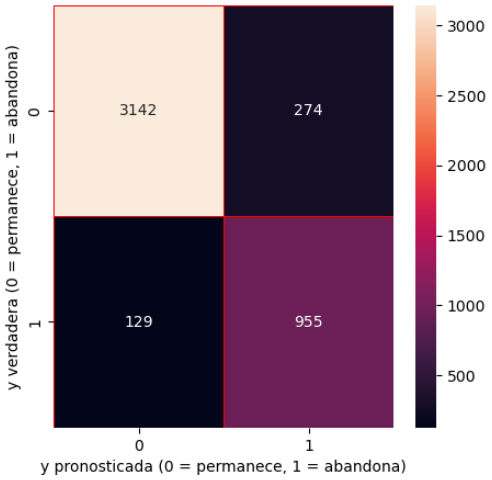
En la primera casilla aparecen 3,142 empleados que realmente permanecieron y que el modelo clasificó correctamente como permanencia. En la segunda aparecen 274 empleados que permanecieron, pero que fueron clasificados equivocadamente como si hubieran abandonado la empresa. En la tercera casilla se encuentran 129 empleados que sí abandonaron la empresa, pero que el modelo clasificó como si hubieran permanecido. Por último, la cuarta casilla contiene 955 empleados que sí abandonaron la empresa y fueron identificados correctamente.

```
# Creación de la matriz de confusión
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_true,y_pred)
cm
```

```
array([[3142,  274],
       [ 129,  955]])
```

```
# Visualización de la matriz de confusión
import seaborn as sns
f, ax = plt.subplots(figsize = (5,5))

sns.heatmap(cm,annot = True, linewidths = 0.5, linecolor = "red", fmt = ".0f", ax = ax)
plt.xlabel("y pronosticada (0 = permanece, 1 = abandona)")
plt.ylabel("y verdadera (0 = permanece, 1 = abandona)")
plt.show()
```



Después se generó la curva ROC para K = 1 siguiendo el código de la parte final de la Lección 6. La gráfica compara la proporción de empleados que abandonaron la empresa y fueron identificados correctamente con la proporción de empleados que permanecieron y fueron clasificados por error como abandonos.

El área bajo la curva, conocida como AUC, fue aproximadamente de 0.90. Esto indica una buena capacidad de separación entre las dos categorías en las observaciones utilizadas para la prueba, aunque poca comparada con la del ejemplo visto en la lección.

```
from sklearn.metrics import roc_curve
from sklearn.metrics import auc

y_scores = knn.predict_proba(x_test)

# fpr = False positive rate, tpr = True positive rate
fpr, tpr, threshold = roc_curve(y_test, y_scores[:,1])
roc_auc = auc(fpr,tpr)

plt.title("Reciever Operating Characteristics")
plt.plot(fpr,tpr,"b", label = "AUC = %0.2f" % roc_auc)
plt.legend(loc = "lower right")
plt.plot([0,1],[0,1], "r--")
plt.xlim([0,1])
plt.ylim([0,1])
plt.ylabel("Tasa de positivos verdaderos")
plt.xlabel("Tasa de falsos positivos")
plt.title("Curva ROC de KNN")
plt.show()
```

